

SDN Project Work: Network Slicing with Ryu and Mininet

Roberta Granata M63001683

Corso di Laurea Magistrale in Ingegneria Informatica

Università degli Studi di Napoli Federico II

23 luglio 2025

Indice

1	Introduzione	3
2	Obiettivi del progetto	3
3	Ambiente di sviluppo e tecnologie	4
4	Descrizione della Topologia	4
5	Implementazione della topologia	5
6	Topology Slicing	7
6.1	Verifica e Test	10
7	Service Slicing	14
7.1	Verifica e test	19
8	Estensione	23
8.1	Sistema di Monitoraggio SDN con Dashboard Web	23
9	Conclusioni	26

1 Introduzione

Negli ultimi anni, il volume di dati che transita attraverso le reti di comunicazione è aumentato in modo esponenziale, spinto dalla diffusione di servizi di streaming, applicazioni mobili e soluzioni basate sull'Internet of Things (IoT). Questo incremento della domanda impone alle infrastrutture di rete di essere non solo scalabili, ma anche altamente flessibili, in grado di adattarsi in tempo reale alle fluttuazioni del traffico e alle crescenti esigenze di qualità del servizio (QoS).

Le architetture di rete tradizionali, seppur affidabili, mostrano spesso limiti significativi in scenari dinamici, dove la capacità di riconfigurare tempestivamente percorsi e priorità diventa essenziale. Per rispondere a queste nuove sfide, sono emerse due tecnologie chiave, progettate per garantire maggiore controllo, programmabilità e adattabilità delle reti moderne.

- **Software-Defined Networking (SDN)**, che separa il piano di controllo (control plane) dal piano di instradamento dei pacchetti (data plane), offrendo un controllo centralizzato e programmabile della rete.
- **Network Slicing**, che consente di creare più reti virtuali (slice) indipendenti all'interno della stessa infrastruttura fisica, ciascuna configurata con parametri di prestazioni e sicurezza dedicati.

All'interno di questo progetto tali tecnologie sono state applicate tramite due strumenti:

- **Mininet**, un emulatore software che permette di ricreare topologie di rete complesse su una singola macchina e di testarle in maniera rapida.
- **Ryu**, un framework per lo sviluppo di controller SDN basati su OpenFlow, scelto per la sua leggerezza e la ricca documentazione disponibile.

Ci concentreremo su due modalità di slicing: *Topology Slicing*, per garantire isolamento e sicurezza tra coppie di host, e *Service Slicing*, per attribuire priorità al traffico video in scenari di congestione. A valle di questa panoramica tecnica, illustreremo la metodologia di implementazione, i test di verifica e i risultati ottenuti, corredati da screenshot e comandi utilizzati.

2 Obiettivi del progetto

L'obiettivo principale di questo progetto è l'implementazione del Network Slicing in un ambiente SDN, utilizzando Mininet come emulatore di rete e Ryu come controller. Il progetto si propone di esplorare e dimostrare due forme fondamentali di slicing:

- **Topology Slicing** Realizzare un'infrastruttura di rete suddivisa logicamente, in cui specifici host possono comunicare solo attraverso percorsi predefiniti. Questo tipo di slicing garantisce isolamento topologico tra le diverse "slice" della rete, migliorando la sicurezza e la gestione del traffico.
- **Service Slicing** Implementare una differenziazione dei flussi di traffico in base al tipo di servizio, dando priorità a quelli ritenuti critici (come il traffico video su UDP/9999). Questo consente una gestione efficiente delle risorse di rete, assicurando Quality of Service (QoS) per le applicazioni più sensibili.

In particolare, gli obiettivi specifici del progetto sono:

- **Progettare una topologia SDN con almeno due slice distinte.**
- **Configurare il controller Ryu per applicare regole di instradamento basate su indirizzi MAC, al fine di realizzare percorsi dedicati tra coppie di host.**
- **Rilevare e classificare il traffico in base al protocollo e alla porta di destinazione, assegnando priorità al traffico video (UDP/9999).**
- **Validare il corretto funzionamento delle slice tramite strumenti di analisi del traffico (Wireshark, TCPDump) e test di throughput (iPerf).**
- **Dimostrare i benefici del network slicing in termini di isolamento, QoS e utilizzo ottimizzato delle risorse di rete**

3 Ambiente di sviluppo e tecnologie

Come già definito dal progetto, gli strumenti utilizzati sono:

- **Mininet**, un emulatore di rete estremamente versatile: in pochi comandi consente di creare topologie complesse, replicando fedelmente switch, host e collegamenti.
- **Ryu**, un framework Python per lo sviluppo di controller SDN basati su OpenFlow.

Entrambi gli strumenti, con la loro comunità attiva e documentazione dettagliata, ci hanno consentito di passare rapidamente dalla teoria alla pratica, focalizzandoci sull'implementazione delle funzionalità di slicing piuttosto che sulla complessità dell'infrastruttura sottostante.

4 Descrizione della Topologia

La topologia di rete implementata è composta da quattro host (H1, H2, H3, H4) e tre switch OpenFlow (S1, S2, S3), gestiti da un controller SDN Ryu. L'architettura è progettata per supportare due slice distinte, con l'obiettivo di dimostrare la funzionalità di topology slicing e di service slicing.

- **Slice superiore:** collega H1 e H3 attraverso il percorso $S1 \rightarrow S2 \rightarrow S4$.
- **Slice inferiore:** collega H2 e H4 tramite un percorso alternativo al primo.

I collegamenti tra switch sono configurati in modo da creare due percorsi logicamente separati per le comunicazioni tra le rispettive coppie di host. Ogni slice è isolata, quindi H1 può comunicare solo con H3 e H2 solo con H4. Tutti gli altri tentativi di comunicazione inter-host sono bloccati tramite regole installate dinamicamente dal controller Ryu, che sfrutta il matching degli indirizzi MAC.

La topologia implementata è quella mostrata in figura:

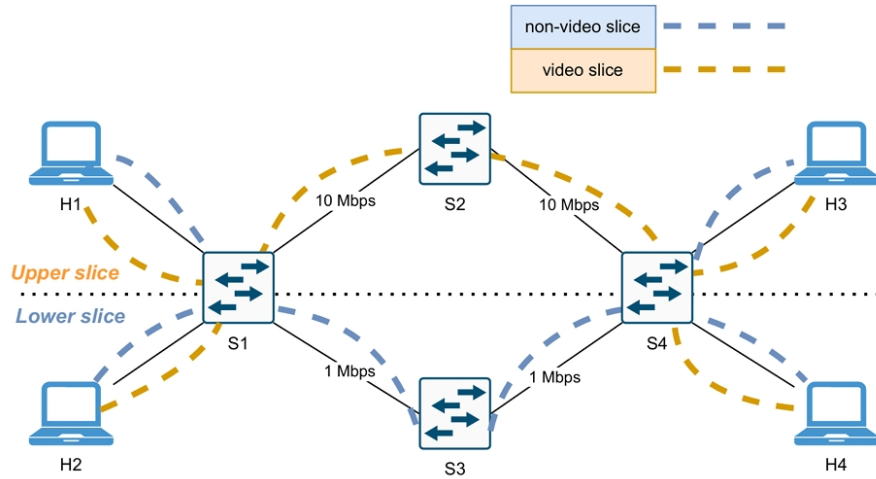


Figura 1: Topologia

Questa struttura consente di testare la gestione del traffico e il comportamento della rete in scenari con segmentazione logica, come quelli tipici di ambienti 5G, edge computing o multi-tenant.

5 Implementazione della topologia

Quindi la prima cosa effettuata è stata l'implementazione della topologia sopra descritta, all'interno dello script `network_topo.py`, scegliendo in modo consapevole la disposizione di host e switch per poter isolare i due flussi di traffico che volevamo sperimentare.

```

1 from mininet.node import OVSKernelSwitch, RemoteController
2 from mininet.cli import CLI
3 from mininet.link import TCLink
4
5
6 class NetworkSlicingTopo(Topo):
7     def __init__(self):
8         Topo.__init__(self)
9
10        host_config = dict(inNamespace=True)
11        http_link_config = dict(bw=1)
12        video_link_config = dict(bw=10)
13        host_link_config = dict()
14
15        for i in range(4):
16            sconfig = {"dpid": "%016x" % (i + 1)}
17            self.addSwitch("s%d" % (i + 1), **sconfig)
18
19        for i in range(4):
20            mac_addr = "00:00:00:00:00:%02x" % (i + 1)
21            self.addHost("h%d" % (i + 1), mac=mac_addr,
22                        **host_config)

```

```

23     self.addLink("s1", "s2", **video_link_config)
24     self.addLink("s2", "s4", **video_link_config)
25     self.addLink("s1", "s3", **http_link_config)
26     self.addLink("s3", "s4", **http_link_config)
27
28     self.addLink("h1", "s1", **host_link_config)
29     self.addLink("h2", "s1", **host_link_config)
30     self.addLink("h3", "s4", **host_link_config)
31     self.addLink("h4", "s4", **host_link_config)
32
33
34     topos = {"networkslicingtopo": (lambda: NetworkSlicingTopo())}
35
36     if __name__ == "__main__":
37         topo = NetworkSlicingTopo()
38         net = Mininet(
39             topo=topo,
40             switch=OVSKernelSwitch,
41             build=False,
42             autoSetMacs=True,
43             autoStaticArp=True,
44             link=TCLink,
45         )
46         controller = RemoteController("c1", ip="127.0.0.1", port=6633)
47         net.addController(controller)
48         net.build()
49         net.start()
50         CLI(net)
51         net.stop()

```

Listing 1: Topologia di rete definita in Mininet

Quindi sono stati definiti i quattro host, ciascuno con un indirizzo MAC univoco, e analogamente sono stati definiti i quattro switch, assegnando a ciascuno un identificativo univoco. Successivamente sono stati creati i collegamenti necessari tra switch e tra host e switch nella topologia, e le proprietà di tali link sono state configurate tramite parametri specifici, per simulare due diversi tipi di traffico.

In particolare, il percorso superiore, composto dai link s1–s2 e s2–s4, è configurato con una banda larga di 10 Mbps. Questo percorso è dedicato al traffico video, che necessita di maggiore capacità e priorità.

Invece, il percorso inferiore, costituito dai link s1–s3 e s3–s4, è configurato con una banda limitata a 1 Mbps, rappresentando un percorso per traffico meno prioritario, come ad esempio quello HTTP.

Per quanto riguarda i collegamenti tra host e switch, gli host h1 e h2 sono collegati allo switch s1, mentre gli host h3 e h4 sono collegati allo switch s4.

I link host-switch non presentano configurazioni specifiche di banda o latenza, utilizzando i parametri di default.

Questa configurazione permette di definire due “slice” di rete distinte, con caratteristiche di capacità differenziate.

La topologia implementata è stata testata su mininet:

```

roberta@roberta-virtual-machine:~/mininet$ sudo python3 network_topo.py
[sudo] password for roberta:
Unable to contact the remote controller at 127.0.0.1:6633
mininet> nodes
available nodes are:
c1 h1 h2 h3 h4 s1 s2 s3 s4
mininet> links
h1-eth0<->s1-eth3 (OK OK)
h2-eth0<->s1-eth4 (OK OK)
h3-eth0<->s4-eth3 (OK OK)
h4-eth0<->s4-eth4 (OK OK)
s1-eth1<->s2-eth1 (OK OK)
s1-eth2<->s3-eth1 (OK OK)
s2-eth2<->s4-eth1 (OK OK)
s3-eth2<->s4-eth2 (OK OK)
mininet> net
h1 h1-eth0:s1-eth3
h2 h2-eth0:s1-eth4
h3 h3-eth0:s4-eth3
h4 h4-eth0:s4-eth4
s1 lo: s1-eth1:s2-eth1 s1-eth2:s3-eth1 s1-eth3:h1-eth0 s1-eth4:h2-eth0
s2 lo: s2-eth1:s1-eth1 s2-eth2:s4-eth1
s3 lo: s3-eth1:s1-eth2 s3-eth2:s4-eth2
s4 lo: s4-eth1:s2-eth2 s4-eth2:s3-eth2 s4-eth3:h3-eth0 s4-eth4:h4-eth0
c1
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=3001>
<Host h2: h2-eth0:10.0.0.2 pid=3003>
<Host h3: h3-eth0:10.0.0.3 pid=3005>
<Host h4: h4-eth0:10.0.0.4 pid=3007>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None,s1-eth4:None pid=3012>
<OVSSwitch s2: lo:127.0.0.1,s2-eth1:None,s2-eth2:None pid=3015>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None pid=3018>
<OVSSwitch s4: lo:127.0.0.1,s4-eth1:None,s4-eth2:None,s4-eth3:None,s4-eth4:None pid=3021>

```

Figura 2: Topologia

6 Topology Slicing

Questa parte del progetto richiedeva nello specifico di realizzare un'infrastruttura di rete suddivisa logicamente, in cui specifici host possono comunicare solo attraverso percorsi predefiniti. In particolare, il controller doveva essere sviluppato in modo da permettere che H1 comunicasse solo con H3 tramite l'upper slice, mentre H2 comunicasse solo con H4 tramite il lower slice.

Per realizzare questo meccanismo, abbiamo modificato il controller SDN Ryu è stato modificato per imporre lo slicing della topologia utilizzando regole di flusso basate sugli indirizzi MAC

```

1 from ryu.base import app_manager
2 from ryu.controller import ofp_event
3 from ryu.controller.handler import CONFIG_DISPATCHER,
  MAIN_DISPATCHER
4 from ryu.controller.handler import set_ev_cls
5 from ryu.ofproto import ofproto_v1_3
6 from ryu.lib.packet import packet
7 from ryu.lib.packet import ethernet
8 from ryu.lib.packet import ether_types
9
10
11 class Controller(app_manager.RyuApp):
12     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
13
14     def __init__(self, *args, **kwargs):
15         super(Controller, self).__init__(*args, **kwargs)
16         self.mac_to_port = {}
17
18         # MAC addresses
19         self.mac_h1 = "00:00:00:00:00:01"

```

```

20     self.mac_h2 = "00:00:00:00:00:02"
21     self.mac_h3 = "00:00:00:00:00:03"
22     self.mac_h4 = "00:00:00:00:00:04"
23
24     # coppie consentite
25     self.allowed_pairs = {
26         (self.mac_h1, self.mac_h3),
27         (self.mac_h3, self.mac_h1),
28         (self.mac_h2, self.mac_h4),
29         (self.mac_h4, self.mac_h2),
30     }
31
32     @set_ev_cls(ofp_event.EventOFPSwitchFeatures,
33                 CONFIG_DISPATCHER)
34     def switch_features_handler(self, ev):
35         datapath = ev.msg.datapath
36         ofproto = datapath.ofproto
37         parser = datapath.ofproto_parser
38
39         match = parser.OFPMatch()
40         actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
41                                           ofproto.OFPCML_NO_BUFFER)]
42         self.add_flow(datapath, 0, match, actions)
43
44     def add_flow(self, datapath, priority, match, actions,
45                 buffer_id=None):
46         ofproto = datapath.ofproto
47         parser = datapath.ofproto_parser
48         inst =
49             [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
50                                           actions)]
51
52         if buffer_id:
53             mod = parser.OFPFlowMod(datapath=datapath,
54                                     buffer_id=buffer_id,
55                                     priority=priority, match=match,
56                                     instructions=inst)
57         else:
58             mod = parser.OFPFlowMod(datapath=datapath,
59                                     priority=priority,
60                                     match=match, instructions=inst)
61         datapath.send_msg(mod)
62
63     @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
64     def _packet_in_handler(self, ev):
65         msg = ev.msg
66         datapath = msg.datapath
67         parser = datapath.ofproto_parser
68         ofproto = datapath.ofproto
69         in_port = msg.match['in_port']
70         dpid = datapath.id

```



```

64
65     pkt = packet.Packet(msg.data)
66     eth = pkt.get_protocols(ethernet.ethernet)[0]
67
68     dst = eth.dst
69     src = eth.src
70
71     if eth.ethertype == ether_types.ETH_TYPE_LLDP:
72         return
73
74     self.mac_to_port.setdefault(dpid, {})
75     self.mac_to_port[dpid][src] = in_port
76
77     # le comunicazioni non autorizzate sono bloccate
78     if (src, dst) not in self.allowed_pairs:
79         self.logger.info("Blocked unauthorized flow: %s ->
80             %s", src, dst)
81         return
82
83     # porta di uscita
84     if dst in self.mac_to_port[dpid]:
85         out_port = self.mac_to_port[dpid][dst]
86     else:
87         out_port = ofproto.OFPP_FLOOD
88
89     actions = [parser.OFPACTIONOutput(out_port)]
90
91     # Install flow to avoid future PacketIn
92     match = parser.OFPMATCH(in_port=in_port, eth_src=src,
93         eth_dst=dst)
94     if msg.buffer_id != ofproto.OFP_NO_BUFFER:
95         self.add_flow(datapath, 1, match, actions,
96             msg.buffer_id)
97         return
98     else:
99         self.add_flow(datapath, 1, match, actions)
100
101     data = None
102     if msg.buffer_id == ofproto.OFP_NO_BUFFER:
103         data = msg.data
104
105     out = parser.OFPPACKETOut(datapath=datapath,
106         buffer_id=msg.buffer_id,
107             in_port=in_port,
108             actions=actions, data=data)
109     datapath.send_msg(out)

```

Listing 2: Topology slicing

Il controller è sviluppato usando Ryu e il protocollo OpenFlow 1.3. Nel costruttore `__init__`, vengono definiti:

- **mac_to_port**: dizionario per apprendere dinamicamente le porte associate ai MAC.
- **allowed_pairs**: insieme di coppie MAC autorizzate alla comunicazione (H1H3, H2H4).

Nel metodo `switch_features_handler`, viene installata una regola di table-miss per inviare i pacchetti non corrispondenti a nessuna regola al controller. Questa regola è fondamentale perché permette al controller di ricevere i pacchetti sconosciuti e decidere cosa farne.

Il metodo `add_flow` crea e invia messaggi `OFPFLOWMod` allo switch per installare regole di forwarding nei datapath. Questo messaggio permette anche di specificare:

- **Match**: le condizioni che un pacchetto deve soddisfare (es. MAC sorgente/destinatario, porta di ingresso).
- **Actions**: cosa fare con i pacchetti che corrispondono (es. inviarli su una certa porta).
- **Priority**: la priorità della regola (più alta = più importante).

In questo modo, una volta che una regola è installata, i pacchetti successivi non vengono più inviati al controller, ma gestiti direttamente dallo switch.

Il metodo `_packet_in_handler` gestisce i pacchetti che arrivano al controller tramite eventi `PacketIn`, ovvero pacchetti che non hanno trovato una regola corrispondente nello switch.

Il comportamento del controller segue i seguenti passaggi:

[label=)]Apprendimento MAC

Il controller registra su quale porta è stato ricevuto un certo indirizzo MAC sorgente (`src`). Questa informazione viene salvata nel dizionario `mac_to_port[dpid][src]` e serve per sapere dove inoltrare i pacchetti di risposta in futuro. **Controllo delle coppie autorizzate**

Il controller verifica se la coppia (`src`, `dst`) è presente nell'insieme `allowed_pairs`. Se non lo è, il pacchetto viene ignorato (bloccato), impedendo comunicazioni non autorizzate. **Determinazione della porta di uscita**

Se la comunicazione è autorizzata e il MAC di destinazione (`dst`) è già stato appreso, il controller recupera la porta associata e prepara l'inoltro del pacchetto.

Installazione del flusso

Per evitare che lo stesso tipo di pacchetto venga inviato nuovamente al controller, viene installata una regola (`flow`) nello switch utilizzando la funzione `add_flow`.

Inoltro del pacchetto

Infine, il pacchetto viene inoltrato direttamente alla porta corretta tramite un messaggio `PacketOut`.

6.1 Verifica e Test

Per verificare la correttezza dell'implementazione di slicing, è stato effettuato un controllo di connettività tramite il comando

```
1 pingall
```

Questo ha mostrato il seguente risultato:

```
roberta@roberta-virtual-machine:~/mininet$ sudo python3 network_topo.py
[sudo] password for roberta:
mininet> pingall
** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
** Results: 66% dropped (4/12 received)
mininet> 
```

Figura 3: Pingall

Questo risultato mostra che H1 comunica solo con H3, e H2 comunica solo con H4, come richiesto. Successivamente sono stati fatti test di connettività più specifici, per analizzare lo scambio di pacchetti tra coppie di host specifici.

```
mininet> h1 ping -c 4 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=1.07 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.111 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.112 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.152 ms

--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3038ms
```

Figura 4: Ping tra H1 e H3

```
mininet> h2 ping -c 4 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=0.966 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=0.152 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.153 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=0.142 ms

--- 10.0.0.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3040ms
rtt min/avg/max/mdev = 0.142/0.353/0.966/0.353 ms
```

Figura 5: Ping tra H2 e H4

```
mininet> h1 ping -c 4 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
^C
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3099ms
```

Figura 6: Ping tra H1 e H2

```
mininet> h4 ping -c 4 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.

--- 10.0.0.3 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3080ms
```

Figura 7: Ping tra H4 e H3

Successivamente, sono state fatte verifiche sulle regole installate su ciascun switch, attraverso il comando

```
1 sh ovs-ofctl dump-flows s
```

Dove s indica lo specifico switch di cui si intende ottenere le regole.

```
mininet> sh ovs-ofctl dump-flows s1
cookie=0x0, duration=989.366s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s1-eth3",dl_dst=00:00:00:00:00:03 actions=out
put:"s1-eth1"
cookie=0x0, duration=989.343s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s1-eth1",dl_dst=00:00:00:00:00:01 actions=out
put:"s1-eth3"
cookie=0x0, duration=959.301s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s1-eth4",dl_dst=00:00:00:00:00:04 actions=out
put:"s1-eth2"
cookie=0x0, duration=959.288s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s1-eth2",dl_dst=00:00:00:00:00:02 actions=out
put:"s1-eth4"
cookie=0x0, duration=1004.933s, table=0, n_packets=71, n_bytes=8267, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl dump-flows s2
cookie=0x0, duration=994.675s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s2-eth1",dl_dst=00:00:00:00:00:03 actions=out
put:"s2-eth2"
cookie=0x0, duration=994.663s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s2-eth2",dl_dst=00:00:00:00:00:01 actions=out
put:"s2-eth1"
cookie=0x0, duration=1010.236s, table=0, n_packets=49, n_bytes=7472, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl dump-flows s3
cookie=0x0, duration=967.726s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s3-eth1",dl_dst=00:00:00:00:00:04 actions=out
put:"s3-eth2"
cookie=0x0, duration=967.718s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s3-eth2",dl_dst=00:00:00:00:00:02 actions=out
put:"s3-eth1"
cookie=0x0, duration=1013.360s, table=0, n_packets=53, n_bytes=7681, priority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl dump-flows s4
cookie=0x0, duration=1000.766s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s4-eth1",dl_dst=00:00:00:00:00:03 actions=ou
tput:"s4-eth3"
cookie=0x0, duration=1000.763s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s4-eth3",dl_dst=00:00:00:00:00:01 actions=ou
tput:"s4-eth1"
cookie=0x0, duration=970.708s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s4-eth2",dl_dst=00:00:00:00:00:04 actions=out
put:"s4-eth4"
cookie=0x0, duration=970.705s, table=0, n_packets=5, n_bytes=490, priority=1,in_port="s4-eth4",dl_dst=00:00:00:00:00:02 actions=out
put:"s4-eth2"
cookie=0x0, duration=1016.345s, table=0, n_packets=83, n_bytes=10308, priority=0 actions=CONTROLLER:65535
```

Figura 8: flows

Per ottenere delle verifiche più efficienti, è stato poi testato il traffico su ciascuno switch, per verificare che i pacchetti passassero per gli slice dedicati. In particolare, per verificare che il traffico tra h1 e h3 utilizzi il percorso superiore, è stato eseguito il seguente comando da Mininet:

```
3. mininet> h1 ping -c 3 h3
```

Contemporaneamente, è stato monitorato il traffico sull'interfaccia s1-eth2, che collega lo switch s1 allo switch s3, utilizzando tcpdump:

```
sudo tcpdump -i s1-eth2
```

L'output di tcpdump ha mostrato pacchetti ICMP in transito, confermando che il traffico tra h1 e h3 passa correttamente attraverso il percorso superiore della rete, come previsto dal comportamento del controller.

```
mininet> xterm s1
mininet> h1 ping -c 3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=9.36 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.143 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.159 ms

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time
rtt min/avg/max/mdev = 0.143/3.220/9.359/4.340 ms
mininet>
```

```
root@roberta-virtual-machine:/home/roberta/mininet# sudo tcpdump -i s2-eth1
tcpdump: verbose output suppressed, use -v(-vv...) for full protocol decode
listening on s2-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
ping13:20:15.291556 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 26705, seq 1, length 64
13:20:15.291800 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 26705, seq 1, length 64
13:20:16.284428 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 26705, seq 2, length 64
13:20:16.284478 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 26705, seq 2, length 64
13:20:17.337069 IP 10.0.0.1 > 10.0.0.3: ICMP echo request, id 26705, seq 3, length 64
13:20:17.337186 IP 10.0.0.3 > 10.0.0.1: ICMP echo reply, id 26705, seq 3, length 64
```

Figura 9: Traffico su S1

Analogamente, eseguendo il comando da Mininet:

```
mininet> h2 ping -c 3 h4
```

e analizzando il traffico su S1, i risultati mostrano che non ci sono pacchetti ICMP in transito, confermando che il traffico tra h2 e h4 non passa attraverso il percorso superiore della rete.

```
mininet> h2 ping -c 3 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=1.10 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=0.109 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.162 ms

--- 10.0.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2048ms
rtt min/avg/max/mdev = 0.109/0.455/1.096/0.453 ms
mininet>
```

```
"Node: s1" (root)
root@roberta-virtual-machine:/home/roberta/mininet# sudo tcpdump -i s1-eth1
tcpdump: verbose output suppressed, use -v(-vv...) for full protocol decode
listening on s1-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

Figura 10: Traffico su S1

Verificando invece la connettività tra H2 e H4 tramite il comando

```
mininet> h2 ping -c 3 h4
```

e contemporaneamente, monitorando il traffico sull'interfaccia s3-eth1, che collega lo switch s3 allo switch s1, utilizzando tcpdump:

```
sudo tcpdump -i s3-eth1
```

L'output mostra i pacchetti ICMP in transito, confermando che in questo caso il traffico tra h2 e h4 passa correttamente attraverso il percorso inferiore della rete.

```
mininet> xterm s3
mininet> h2 ping -c 3 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=0.925 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=0.108 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.114 ms

--- 10.0.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2019ms
rtt min/avg/max/mdev = 0.108/0.382/0.925/0.383 ms
mininet>
```

```
"Node: s3" (root)
root@roberta-virtual-machine:/home/roberta/mininet# sudo tcpdump -i s3-eth1
tcpdump: verbose output suppressed, use -v(-vv...) for full protocol decode
listening on s3-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:23:59.639281 IP 10.0.0.4 > 10.0.0.2: ICMP echo request, id 58559, seq 1, length 64
13:24:00.640195 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 58559, seq 2, length 64
13:24:00.640239 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 58559, seq 2, length 64
13:24:01.657039 IP 10.0.0.2 > 10.0.0.4: ICMP echo request, id 58559, seq 3, length 64
13:24:01.657084 IP 10.0.0.4 > 10.0.0.2: ICMP echo reply, id 58559, seq 3, length 64
```

Figura 11: Traffico su S3

7 Service Slicing

La seconda parte del progetto prevedeva di implementare un **traffic-based slicing** nel controller SDN con Ryu, in modo da identificare e dare priorità al traffico video, distinguendolo da altri tipi di traffico. In questo caso, il traffico video è definito come UDP diretto alla porta 9999 (es. streaming video simulato con iPerf o simili).

"Trafficbased slicing" consiste nel classificare i flussi di rete in base a caratteristiche del traffico e applicare regole diverse a ciascuna classe di traffico.

Analogamente per il topology slicing, è stato modificato il controller, in modo da ottenere il comportamento desiderato.

```
1 from ryu.base import app_manager
2 from ryu.controller import ofp_event
3 from ryu.controller.handler import
    CONFIG_DISPATCHER, MAIN_DISPATCHER
4 from ryu.controller.handler import set_ev_cls
5 from ryu.ofproto import ofproto_v1_3
6 from ryu.lib.packet import packet
7 from ryu.lib.packet import ethernet
8 from ryu.lib.packet import ether_types
9 from ryu.lib.packet import udp
10 from ryu.lib.packet import tcp
11 from ryu.lib.packet import icmp
12
13
14 class ServiceSlicing(app_manager.RyuApp):
15     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
16
17     def __init__(self, *args, **kwargs):
18         super(ServiceSlicing, self).__init__(*args,
19                                             **kwargs)
20
21         # outport =
22         self.mac_to_port[dpid][mac_address]
23         self.mac_to_port = {
24             1: {"00:00:00:00:00:01": 3,
25                "00:00:00:00:00:02": 4},
26             4: {"00:00:00:00:00:03": 3,
27                "00:00:00:00:00:04": 4},
28         }
29         self.slice_TCport = 9999
30
31         # outport =
32         self.slice_ports[dpid][slicenumber]
33         self.slice_ports = {1: {1: 1, 2: 2}, 4: {1:
34             1, 2: 2}}
35         self.end_swatches = [1, 4]
36
37     @set_ev_cls(ofp_event.EventOFPSwitchFeatures,
```

```

CONFIG_DISPATCHER)
32 def switch_features_handler(self, ev):
33     datapath = ev.msg.datapath
34     ofproto = datapath.ofproto
35     parser = datapath.ofproto_parser
36
37     # install the table-miss flow entry.
38     match = parser.OFPMatch()
39     actions = [
40         parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
41                                ofproto.OFPCML_NO_BUFFER)
42     ]
43     self.add_flow(datapath, 0, match, actions)
44
45 def add_flow(self, datapath, priority, match,
46             actions):
47     ofproto = datapath.ofproto
48     parser = datapath.ofproto_parser
49
50     # construct flow_mod message and send it.
51     inst =
52         [parser.OFPInstructionActions(ofproto.OFPI_APPLY_ACTIONS,
53                                       actions)]
54     mod = parser.OFPFlowMod(
55         datapath=datapath, priority=priority,
56         match=match, instructions=inst
57     )
58     datapath.send_msg(mod)
59
60 def _send_package(self, msg, datapath, in_port,
61                 actions):
62     data = None
63     ofproto = datapath.ofproto
64     if msg.buffer_id == ofproto.OFP_NO_BUFFER:
65         data = msg.data
66
67     out = datapath.ofproto_parser.OFPPacketOut(
68         datapath=datapath,
69         buffer_id=msg.buffer_id,
70         in_port=in_port,
71         actions=actions,
72         data=data,
73     )
74     datapath.send_msg(out)
75
76 @set_ev_cls(ofp_event.EventOFPPacketIn,
77             MAIN_DISPATCHER)
78 def _packet_in_handler(self, ev):
79     msg = ev.msg
80     datapath = msg.datapath
81     ofproto = datapath.ofproto

```

```

75     in_port = msg.match["in_port"]
76
77     pkt = packet.Packet(msg.data)
78     eth = pkt.get_protocol(ethernet.ethernet)
79
80     if eth.ethertype ==
81         ether_types.ETH_TYPE_LLDP:
82         # ignore lldp packet
83         return
84     dst = eth.dst
85     src = eth.src
86
87     dpid = datapath.id
88
89     if dpid in self.mac_to_port:
90         if dst in self.mac_to_port[dpid]:
91             out_port =
92                 self.mac_to_port[dpid][dst]
93             actions =
94                 [datapath.ofproto_parser.OFPActionOutput(out.
95
96             match =
97                 datapath.ofproto_parser.OFPMatch(eth_dst=dst,
98                 self.add_flow(datapath, 1, match,
99                     actions)
100             self._send_package(msg, datapath,
101                 in_port, actions)
102
103         elif (pkt.get_protocol(udp.udp) and
104             pkt.get_protocol(udp.udp).dst_port
105             == self.slice_TCport):
106             slice_number = 1
107             out_port =
108                 self.slice_ports[dpid][slice_number]
109             match =
110                 datapath.ofproto_parser.OFPMatch(
111                     in_port=in_port,
112                     eth_dst=dst,
113                     eth_type=ether_types.ETH_TYPE_IP,
114                     ip_proto=0x11, # udp
115                     udp_dst=self.slice_TCport,
116                 )
117
118             actions =
119                 [datapath.ofproto_parser.OFPActionOutput(out.
120             self.add_flow(datapath, 2, match,
121                 actions)
122             self._send_package(msg, datapath,
123                 in_port, actions)
124
125         elif (pkt.get_protocol(udp.udp) and
126             pkt.get_protocol(udp.udp).dst_port

```



```

112         != self.slice_TCport):
113             slice_number = 2
114             out_port =
115                 self.slice_ports[dpid][slice_number]
116             match =
117                 datapath.ofproto_parser.OFPMatch(
118                     in_port=in_port,
119                     eth_dst=dst,
120                     eth_src=src,
121                     eth_type=ether_types.ETH_TYPE_IP,
122                     ip_proto=0x11, # udp
123                     udp_dst=pkt.get_protocol(udp.udp).dst_port,
124                 )
125             actions =
126                 [datapath.ofproto_parser.OFPActionOutput(out.
127 self.add_flow(datapath, 1, match,
128                 actions)
129 self._send_package(msg, datapath,
130                 in_port, actions)
131
132 elif pkt.get_protocol(tcp.tcp):
133     slice_number = 2
134     out_port =
135         self.slice_ports[dpid][slice_number]
136     match =
137         datapath.ofproto_parser.OFPMatch(
138             in_port=in_port,
139             eth_dst=dst,
140             eth_src=src,
141             eth_type=ether_types.ETH_TYPE_IP,
142             ip_proto=0x06, # tcp
143         )
144     actions =
145         [datapath.ofproto_parser.OFPActionOutput(out.
146 self.add_flow(datapath, 1, match,
147                 actions)
148 self._send_package(msg, datapath,
149                 in_port, actions)
150
151 elif pkt.get_protocol icmp.icmp):
152     slice_number = 2
153     out_port =
154         self.slice_ports[dpid][slice_number]
155     match =
156         datapath.ofproto_parser.OFPMatch(
157             in_port=in_port,
158             eth_dst=dst,
159             eth_src=src,
160             eth_type=ether_types.ETH_TYPE_IP,
161             ip_proto=0x01, # icmp
162         )

```

```

150         actions =
151             [datapath.ofproto_parser.OFPActionOutput(out_port)]
152         self.add_flow(datapath, 1, match, actions)
153         self._send_package(msg, datapath, in_port, actions)
154
155     elif dpid not in self.end_switches:
156         out_port = ofproto.OFPP_FLOOD
157         actions =
158             [datapath.ofproto_parser.OFPActionOutput(out_port)]
159         match =
160             datapath.ofproto_parser.OFPMatch(in_port=in_port)
161         self.add_flow(datapath, 1, match, actions)
162         self._send_package(msg, datapath, in_port, actions)

```

Listing 3: Service slicing

Nel costruttore `__init__`, vengono inizializzate le seguenti strutture dati:

- **mac_to_port**: dizionario statico che associa indirizzi MAC alle porte di uscita per gli switch finali (ID 1 e 4). Serve per determinare la porta corretta verso un host noto.
- **slice_TCport**: definisce la porta UDP 9999, utilizzata per identificare il traffico video.
- **slice_ports**: mappa che associa ogni slice (1 o 2) a una porta di uscita per ciascun switch.
- **end_switches**: lista degli switch terminali (1 e 4), ovvero quelli direttamente connessi agli host, dove viene applicata la logica di slicing.

Per quanto riguarda invece la gestione degli switch, quando uno switch si connette al controller, viene installata una regola di **table-miss** che inoltra al controller tutti i pacchetti che non corrispondono ad alcuna regola esistente nella tabella di flusso. Questo permette al controller di analizzare e gestire dinamicamente i pacchetti sconosciuti.

Il metodo `add_flow` crea e invia un messaggio `OFPFlowMod` allo switch per installare una nuova regola di forwarding. Ogni regola specifica:

- Le condizioni di **match** (es. tipo di protocollo, porta UDP, indirizzi MAC).
- Le **azioni** da eseguire (es. inoltra su una porta specifica).
- La **priorità** della regola, che determina l'ordine di valutazione.

Il metodo (`_packet_in_handler`) gestisce i pacchetti ricevuti dal controller tramite eventi `PacketIn` e applica la logica di slicing in base al tipo di traffico:

- **MAC noto**: se il pacchetto è destinato a un MAC presente in `mac_to_port`, viene inoltrato direttamente alla porta corrispondente.

- **Traffico video (UDP sulla porta 9999):** viene classificato come traffico prioritario e instradato sulla **slice 1**, utilizzando la porta associata a tale slice nello switch corrente.
- **Traffico UDP generico (diverso da 9999):** viene instradato sulla **slice 2**, considerata come percorso standard.
- **Traffico TCP o ICMP:** anch'esso viene instradato sulla **slice 2**, seguendo la logica di separazione dei servizi.
- **Switch non terminali:** se il pacchetto arriva da uno switch non presente in `end_switches`, viene semplicemente **floodato**, ovvero inviato su tutte le porte, per garantire la propagazione iniziale.

7.1 Verifica e test

Per verificare il corretto funzionamento del controller sono state eseguiti test di varia natura. Al di là dei semplici test di connettività tramite ping, sono stati effettuati controlli volti a verificare che il controller riconoscesse in modo corretto il traffico e lo instradasse di conseguenza. In primis, si è inviato del traffico da H1 a H3, simulando client-server in UDP, usando i seguenti comandi:

```
1 iperf -s -u -p 9999
```

su H3 e il comando:

```
1 iperf -c 10.0.0.3 -u -p 9999 -b 10M
```

The screenshot shows a terminal window with the following content:

```

roberta@roberta-virtual-machine: ~/mininet
roberta@roberta-virtual-machine: ~/mininet
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([0-9a-z:]+-eth[0-9a-z:]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel:Ethernet
rm -f ~/.ssh/mn/*
*** Cleanup complete.
roberta@roberta-virtual-machine: ~/mininet$ sudo python3 network_topo.py
mininet>
mininet>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h2 h3
h3 -> h1 h2 h3
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet> xterm h1 h3
mininet>

```

Two terminal windows are also shown in the foreground, titled "Node: h1" and "Node: h3".

Node: h1 output:

```

Client connecting to 10.0.0.3, UDP port 9999
Sending 1470 byte datagrams, IPG target: 1121.52 us (hainan adjust)
UDP buffer size: 208 KByte (default)
[ 1] local 10.0.0.1 port 45013 connected with 10.0.0.3 port 9999
[ 0] Interval Transfer Bandwidth
[ 1] 0.0000-1.0000 sec 1.25 MBytes 10.5 Mbits/sec
[ 2] 1.0000-2.0000 sec 1.17 MBytes 9.50 Mbits/sec
[ 3] 2.0000-3.0000 sec 1.13 MBytes 9.49 Mbits/sec
[ 4] 3.0000-4.0000 sec 1.14 MBytes 9.54 Mbits/sec
[ 5] 4.0000-5.0000 sec 1.00 MBytes 8.42 Mbits/sec
[ 6] 5.0000-6.0000 sec 1.11 MBytes 9.34 Mbits/sec
[ 7] 6.0000-7.0000 sec 1.15 MBytes 9.63 Mbits/sec
[ 8] 7.0000-8.0000 sec 1.13 MBytes 9.48 Mbits/sec
[ 9] 8.0000-9.0000 sec 1.13 MBytes 9.51 Mbits/sec
[10] 9.0000-10.0000 sec 1.14 MBytes 9.53 Mbits/sec
[11] 0.0000-10.0055 sec 11.4 MBytes 9.52 Mbits/sec
[12] Sent 8102 datagrams
[13] Server Report:
[14] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[15] 0.0000-10.0515 sec 11.4 MBytes 9.48 Mbits/sec 4.533 ms 0/8101 (0%)
[16] 0.0000-10.0515 sec 12 datagrams received out-of-order
root@roberta-virtual-machine:/home/roberta/mininet#

```

Node: h3 output:

```

root@roberta-virtual-machine:/home/roberta/mininet# iperf -s -u -p 9999 -b 10M
Server listening on UDP port 9999
UDP buffer size: 208 KByte (default)
[ 1] local 10.0.0.3 port 9999 connected with 10.0.0.1 port 45013
[ 0] Interval Transfer Bandwidth Jitter Lost/Total Datagrams
[ 1] 0.0000-10.0515 sec 11.4 MBytes 9.48 Mbits/sec 4.534 ms 0/8101 (0%)
[ 2] 0.0000-10.0515 sec 12 datagrams received out-of-order
[ 3] WARNING: ack of last datagram failed.

```

Figura 12: Traffico tra H1 e H3

Il risultato dimostra che il controller riconosce il traffico video e lo trasmette verso un percorso prioritario ad alta banda. Si può notare che la banda di trasmissione è vicina ai 10 Mbs.

Analogamente, si può simulare traffico TCP, che non è riconosciuto come priorità. Quindi:

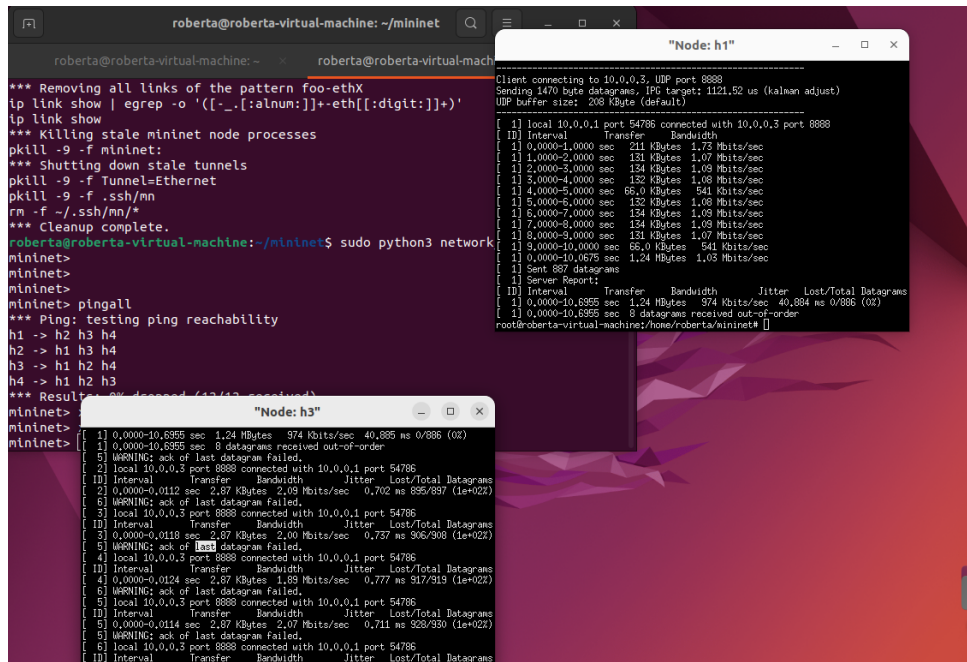


Figura 15: Traffico UDP tra H1 e H3

Altro tipo di test è stato effettuato sfruttando il comando:

```
1 sh ovs-ofctl dump-flows s1
```

che permette di visualizzare tutti i flussi installati nello switch s1, in questo caso.

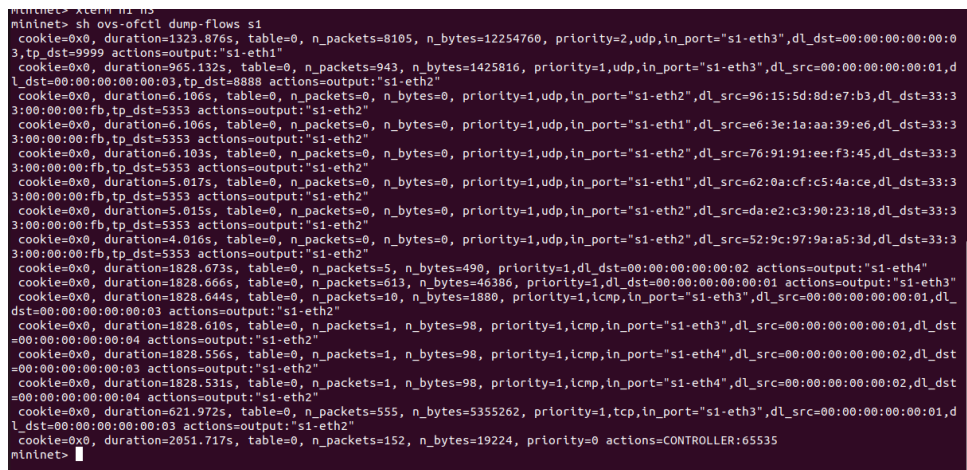


Figura 16: flows-dump S1

Interpretazione delle regole di flusso

Regola 1

cookie=0x0, duration=1323.876s, table=0, n_packets=8105, n_bytes=12254760, priority=2, udp, in_port="s1-eth3", d1_dst=00:00:00:00:00:03, tp_dst=9999 actions=output:"s1-eth1"

Questa regola ha **priorità 2** (più alta). Matcha pacchetti **UDP** in ingresso da **s1-eth3**, con destinazione MAC **00:00:00:00:00:03** e porta UDP di destinazione **9999**. L'azione è inoltrare i pacchetti sull'interfaccia **s1-eth1**. Le statistiche mostrano che sono passati **8105**

pacchetti per un totale di circa **12 MB** di dati. Questa è con tutta probabilità la regola della **video traffic slice**.

Regola 2

```
cookie=0x0, duration=965.132s, table=0, n_packets=943, n_bytes=1425816,  
priority=1, udp, in_port="s1-eth3", dl_src=00:00:00:00:00:01,  
dl_dst=00:00:00:00:00:03, tp_dst=8888  
actions=output:"s1-eth2"
```

Questa regola ha **priorità 1** (meno alta). Matcha pacchetti UDP sulla porta 8888 (ad esempio traffico di test o slice “normale”). I pacchetti vanno da **s1-eth3** a **s1-eth2**. Il numero di pacchetti e byte è inferiore rispetto alla regola precedente. Si tratta probabilmente di traffico non prioritario o appartenente a una slice con banda minore.

Regola 3

```
cookie=0x0, duration=621.972s, table=0, n_packets=555, n_bytes=5355262,  
priority=1, tcp, in_port="s1-eth3", dl_src=00:00:00:00:00:01,  
dl_dst=00:00:00:00:00:03  
actions=output:"s1-eth2"
```

Regola per traffico **TCP**, con **priorità 1**. Il traffico passa da **s1-eth3** a **s1-eth2**. Anche questa è probabilmente associata a una slice a banda più bassa, utilizzata per traffico “normale”.

Conclusione: il controller sta installando flussi con priorità diverse in base al tipo di traffico. Il traffico **UDP video** (porta 9999) viene instradato su un’interfaccia diversa (**s1-eth1**), che rappresenta una slice dedicata con priorità maggiore. Il traffico su altre porte (come UDP 8888 o TCP generico) è instradato su **s1-eth2**, probabilmente appartenente a una slice best-effort. Le statistiche (**n_packets**, **n_bytes**) confermano che le regole sono attive e che il traffico è effettivamente transitato.

Analogamente per S2:

```
mininet> sh ovs-ofctl dump-flows s2  
cookie=0x0, duration=2271.210s, table=0, n_packets=24, n_bytes=2687, priority=1, in_port="s2-eth2" actions=FLOOD  
cookie=0x0, duration=2270.964s, table=0, n_packets=8126, n_bytes=12258189, priority=1, in_port="s2-eth1" actions=FLOOD  
cookie=0x0, duration=2271.453s, table=0, n_packets=1, n_bytes=217, priority=0 actions=CONTROLLER:65535  
mininet> sh ovs-ofctl dump-flows s3  
cookie=0x0, duration=2290.006s, table=0, n_packets=674, n_bytes=53714, priority=1, in_port="s3-eth2" actions=FLOOD  
cookie=0x0, duration=2289.990s, table=0, n_packets=1564, n_bytes=6790696, priority=1, in_port="s3-eth1" actions=FLOOD  
cookie=0x0, duration=2290.258s, table=0, n_packets=2, n_bytes=458, priority=0 actions=CONTROLLER:65535
```

Figura 17: flows-dump S2

Questo mostra che gli switch s2 e s3 mostrano flow con **priority=1** e azione **FLOOD**, ovvero il pacchetto viene inviato su tutte le porte eccetto quella di ingresso. Questo indica che il traffico che arriva su quella porta non ha un flow specifico di forwarding e quindi viene flooding broadcast.

Inoltre, c’è sempre una flow con **priority=0** e azione **CONTROLLER:65535**. I pacchetti vengono inviati al controller e serve per far decidere al controller come trattare pacchetti non riconosciuti o nuovi.

Infine, le flows di S4:

```

mininet> sh on/off/dump/flows.sh
cookie=0x duration=1558.171s, table=0, n_packets=1, n_bytes=170, priority=1,udp,in_port="s4-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01,tp_dst=4001 actions=output:"s4-eth2"
cookie=0x duration=1598.799s, table=0, n_packets=46, n_bytes=8266, priority=1,udp,in_port="s4-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01,tp_dst=54786 actions=output:"s4-eth2"
cookie=0x duration=159.477s, table=0, n_packets=0, n_bytes=0, priority=1,udp,in_port="s4-eth3",dl_src=76:91:91:ee:f3:45,dl_dst=33:33:00:00:00:00,tp_dst=5353 actions=output:"s4-eth2"
cookie=0x duration=256.470s, table=0, n_packets=0, n_bytes=0, priority=1,udp,in_port="s4-eth3",dl_src=76:91:91:ee:f3:45,dl_dst=33:33:00:00:00:00,tp_dst=5353 actions=output:"s4-eth2"
cookie=0x duration=156.463s, table=0, n_packets=0, n_bytes=0, priority=1,udp,in_port="s4-eth3",dl_src=00:00:00:00:00:00,dl_dst=33:33:00:00:00:00,tp_dst=5353 actions=output:"s4-eth2"
cookie=0x duration=249.382s, table=0, n_packets=0, n_bytes=0, priority=1,udp,in_port="s4-eth3",dl_src=00:00:00:00:00:00,dl_dst=33:33:00:00:00:00,tp_dst=5353 actions=output:"s4-eth2"
cookie=0x duration=249.372s, table=0, n_packets=0, n_bytes=0, priority=1,udp,in_port="s4-eth3",dl_src=00:00:00:00:00:00,dl_dst=33:33:00:00:00:00,tp_dst=5353 actions=output:"s4-eth2"
cookie=0x duration=249.377s, table=0, n_packets=0, n_bytes=0, priority=1,udp,in_port="s4-eth3",dl_src=00:00:00:00:00:00,dl_dst=33:33:00:00:00:00,tp_dst=5353 actions=output:"s4-eth2"
cookie=0x duration=2073.003s, table=0, n_packets=0, n_bytes=0, priority=1,dl_dst=00:00:00:00:00:00 actions=output:"s4-eth3"
cookie=0x duration=2072.969s, table=0, n_packets=0, n_bytes=0, priority=1,dl_dst=00:00:00:00:00:00 actions=output:"s4-eth3"
cookie=0x duration=2072.991s, table=0, n_packets=1, n_bytes=99, priority=1,icmp,in_port="s4-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s4-eth2"
cookie=0x duration=2072.959s, table=0, n_packets=1, n_bytes=99, priority=1,icmp,in_port="s4-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s4-eth2"
cookie=0x duration=2072.915s, table=0, n_packets=1, n_bytes=99, priority=1,icmp,in_port="s4-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02 actions=output:"s4-eth2"
cookie=0x duration=2072.889s, table=0, n_packets=1, n_bytes=99, priority=1,icmp,in_port="s4-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02 actions=output:"s4-eth2"
cookie=0x duration=2066.319s, table=0, n_packets=252, n_bytes=36472, priority=1,top,in_port="s4-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s4-eth2"
cookie=0x duration=2250.076s, table=0, n_packets=161, n_bytes=1769, priority=0 actions=controller:65535
mininet>

```

Figura 18: flows-dump S4

Il comportamento osservato è coerente con il controller: su switch 1 e 4 si osservano flow specifici con priorità 1 o 2 che corrispondono alle regole di slicing, mentre su switch 2 e 3 si osservano solo flooding perché il controller li tratta come switch di transito senza regole specifiche.

8 Estensione

8.1 Sistema di Monitoraggio SDN con Dashboard Web

Il controller è stato modificato per includere funzionalità di monitoraggio avanzato. In particolare per ogni flusso rilevato, il controller registra una riga contenente informazioni come MAC sorgente e destinazione, protocollo, porta, switch coinvolto, bitrate, durata e dimensione media dei pacchetti. Per le metriche al posto di valori statici, i dati vengono simulati tramite generatori casuali in Python, che permettono di simulare una rete dinamica e osservare comportamenti realistici attraverso la dashboard.

```

def _log_flow_stats(self, src_mac, dst_mac, proto, dst_port, dpid):
    timestamp = time.time()
    bitrate = random.uniform(3e6, 15e6)      # tra 3 e 15 Mbps
    duration = random.uniform(1, 10)         # tra 1 e 10 secondi
    avg_pkt_size = random.choice([512, 768, 1024, 1400]) # dinamico

    with open("flow_stats.csv", "a") as f:
        f.write(f"{timestamp},{src_mac},{dst_mac},{proto},{dst_port},{dpid},{bitrate}/
1e6:.2f},{duration:.2f},{avg_pkt_size}\n")

```

Figura 19: estensione codice

Questa funzione `_log_flow_stats(...)` ha il compito di registrare le metriche principali di ogni flusso di rete intercettato dal controller SDN all'interno di un file di testo denominato `flow_stats.csv`. Ogni riga del file rappresenta un singolo flusso rilevato. In particolare:

- **Generazione timestamp:** indica il momento esatto in cui il flusso è stato rilevato, tramite la funzione `time.time()`.
- **Simulazione del bitrate:** genera un valore casuale compreso tra 3 e 15 Mbps, per simulare l'ampiezza del traffico.
- **Simulazione durata del flusso:** crea una durata variabile tra 1 e 10 secondi, rappresentando la persistenza del flusso.
- **Dimensione media dei pacchetti:** selezionata casualmente da una lista predefinita di valori (512, 768, 1024, 1400 byte), al fine di riflettere tipologie diverse di traffico.

I dati generati vengono salvati in append nel file `flow_stats.csv`. Questo file è successivamente utilizzato dalla dashboard web per la visualizzazione dei flussi.

Per visualizzare i flussi registrati dal controller, infatti è stata implementata una dashboard web locale tramite Flask. La dashboard è stata creata definendo uno script `app.py`, che costituisce il server centrale dell'applicazione web e permette di leggere i dati dal file `flow_stats.csv` e renderli all'interno di una pagina HTML dinamica.

```
1 from flask import Flask, render_template
2 import pandas as pd
3
4 app = Flask(__name__)
5
6 def load_stats():
7     df = pd.read_csv("flow_stats.csv", names=[
8         "timestamp", "src_mac", "dst_mac", "proto", "dst_port",
9         "switch_id", "bitrate", "duration", "avg_pkt_size"
10    ])
11     return df.tail(50) # Ultimi 50 flussi
12
13 @app.route("/")
14 def dashboard():
15     df = load_stats()
16     data = df.to_dict(orient="records")
17     return render_template("dashboard.html", records=data)
18
19
20 if __name__ == "__main__":
21     print("Flask sta partendo...")
22     app.run(debug=True)
23
```

Figura 20: `app.py`

- **Avvio del server Flask:** tramite la chiamata `app.run(debug=True)`, il sistema avvia un server locale raggiungibile da browser all'indirizzo `http://localhost:5000`.
- **Lettura del file CSV:** la funzione `load_stats()` utilizza la libreria Pandas per leggere gli ultimi 50 flussi registrati. I dati vengono convertiti in formato `dict` per essere facilmente interpretati dalla pagina HTML.
- **Rendering dinamico della pagina:** la funzione `render_template()` serve per generare una pagina web basata sul file `dashboard.html`.

Il file `app.py` in sostanza funge da ponte tra i dati generati dal controller SDN e l'interfaccia visiva.

Successivamente, è stato poi definito un template visivo della dashboard SDN nel file `dashboard.html`. La pagina è progettata per visualizzare in modo interattivo i dati dei flussi registrati dal controller SDN.


```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>SDN Dashboard</title>
5 <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
6 <style>
7 table {border-collapse: collapse; width: 100%;}
8 th, td {border: 1px solid #aaa; padding: 5px;}
9 th {background-color: #f2f2f2;}
10 </style>
11 </head>
12 <body>
13 <h2> Flussi di rete SDN</h2>
14 <table>
15 <tr>
16 <th>Timestamp</th><th>Src MAC</th><th>Dst MAC</th><th>Proto</th>
17 <th>Dst Port</th><th>Switch</th><th>Bitrate (Mbps)</th>
18 <th>Durata (s)</th><th>Pacchetto Medio (B)</th></tr>
19 </tr>
20 <tr>
21 <th colspan="8"> [% for row in records %]
22 <tr>
23 <td>[% row.timestamp %]</td><td>[% row.src_mac %]</td><td>[% row.dst_mac %]</td><td>[% row.proto %]</td>
24 <td>[% row.dst_port %]</td><td>[% row.switch_id %]</td><td>[% row.bitrate %]</td>
25 <td>[% row.duration %]</td><td>[% row.avg_pkt_size %]</td>
26 </tr>
27 <tr>
28 <th colspan="8"> [% endfor %]
29 </table>
30
31 <h3> Grafico Bitrate</h3>
32 <canvas id="myChart"></canvas>
33 <script>
34 const ctx = document.getElementById('myChart').getContext('2d');
35 const chart = new Chart(ctx, {
36 type: 'line',
37 data: {
38 labels: [% for row in records %]{% loop.index %},{% endfor %}],
39 datasets: [{
40 label: 'Bitrate (Mbps)',
41 data: [% for row in records %]{% row.bitrate %},{% endfor %}],
42 borderColor: 'blue',
43 fill: false
44 }, {
45 label: 'Durata (s)',
46 data: [% for row in records %]{% row.duration %},{% endfor %}],
47 borderColor: 'orange',
48 fill: false
49 }, {
50 label: 'Pacchetto Medio (B)',
51 data: [% for row in records %]{% row.avg_pkt_size %},{% endfor %}],
52 borderColor: 'green',
53 fill: false
54 }
55 ]},
56 options: { responsive: true }
57 });
58 </script>
59 </body>
60 </html>

```

Figura 21: Dashboard

- **Visualizzazione tabellare:** mediante un ciclo `% for row in records`, il template mostra gli ultimi 50 flussi di rete, ciascuno con informazioni quali timestamp, indirizzi MAC sorgente e destinazione, protocollo, porta di destinazione, switch ID, bitrate, durata e dimensione media dei pacchetti.
- **Grafico interattivo:** con il supporto della libreria Chart.js integrata da CDN, viene generato un grafico a linee che rappresenta tre metriche chiave:
 - **Bitrate (Mbps)** — rappresenta la velocità simulata del flusso
 - **Durata (s)** — mostra la persistenza del flusso
 - **Pacchetto medio (B)** — indica la dimensione media dei pacchetti

Flussi di rete SDN									
Timestamp	Src MAC	Dst MAC	Proto	Dst Port	Switch	Bitrate (Mbps)	Durata (s)	Pacchetto Medio (B)	
1753286815.026424	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	6.54	4.4	1024.0	
1753286815.206805	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	5.92	2.72	1400.0	
1753286815.432666	3e-f3-99-7f-4a-63	33-33-00-00-00-0b	udp	5353	4	10.66	8.44	768.0	
1753286815.87994	ea-a8-95-89-ce-e8	33-33-00-00-00-0b	udp	5353	1	5.48	4.29	1024.0	
1753286815.8836255	d6-56-07-be-e5-30	33-33-00-00-00-0b	udp	5353	1	14.4	2.23	1024.0	
1753286816.1987522	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	5.22	5.27	1024.0	
1753286816.4404397	3e-f3-99-7f-4a-63	33-33-00-00-00-0b	udp	5353	4	14.9	4.81	1024.0	
1753286817.2094097	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	6.17	6.38	1024.0	
1753286817.602035	3e-f3-99-7f-4a-63	33-33-00-00-00-0b	udp	5353	4	9.19	4.61	512.0	
1753286818.3686752	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	9.11	4.71	512.0	
1753286818.7186503	ea-a8-95-89-ce-e8	33-33-00-00-00-0b	udp	5353	1	10.83	6.78	1024.0	
1753286818.7197816	d6-56-07-be-e5-30	33-33-00-00-00-0b	udp	5353	1	14.81	4.86	768.0	
1753286820.4416986	3e-f3-99-7f-4a-63	33-33-00-00-00-0b	udp	5353	4	4.5	3.31	1024.0	
1753286821.2082915	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	7.45	8.13	512.0	
1753286826.7198808	ea-a8-95-89-ce-e8	33-33-00-00-00-0b	udp	5353	1	10.45	1.64	1400.0	
1753286826.722701	d6-56-07-be-e5-30	33-33-00-00-00-0b	udp	5353	1	13.71	4.37	768.0	
1753286828.444275	3e-f3-99-7f-4a-63	33-33-00-00-00-0b	udp	5353	4	5.3	7.89	512.0	
1753286829.2092943	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	9.57	9.34	512.0	
1753286842.7214615	ea-a8-95-89-ce-e8	33-33-00-00-00-0b	udp	5353	1	3.35	1.19	768.0	
1753286842.722867	d6-56-07-be-e5-30	33-33-00-00-00-0b	udp	5353	1	14.41	6.88	512.0	
1753286844.4449198	3e-f3-99-7f-4a-63	33-33-00-00-00-0b	udp	5353	4	7.71	8.26	512.0	
1753286845.2122276	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	7.31	1.14	512.0	
1753286874.7252097	ea-a8-95-89-ce-e8	33-33-00-00-00-0b	udp	5353	1	8.6	1.23	512.0	
1753286874.7263937	d6-56-07-be-e5-30	33-33-00-00-00-0b	udp	5353	1	14.82	4.46	512.0	
1753286876.448063	3e-f3-99-7f-4a-63	33-33-00-00-00-0b	udp	5353	4	4.98	6.35	512.0	
1753286877.2156825	92-02-37-a3-8b-b9	33-33-00-00-00-0b	udp	5353	4	12.14	1.12	1024.0	
1753286902.200067	00-00-00-00-00-01	00-00-00-00-00-02	mac	-	1	3.85	2.09	768.0	
1753286902.2049153	00-00-00-00-00-02	00-00-00-00-00-01	mac	-	1	9.24	3.68	512.0	
1753286902.2170243	00-00-00-00-00-01	00-00-00-00-00-03	icmp	-	1	7.17	9.42	1024.0	
1753286912.222271	00-00-00-00-00-01	00-00-00-00-00-04	icmp	-	1	13.81	5.1	1024.0	
1753286922.234086	00-00-00-00-00-02	00-00-00-00-00-03	icmp	-	1	10.74	4.25	1024.0	
1753286932.2477455	00-00-00-00-00-02	00-00-00-00-00-04	icmp	-	1	4.69	4.66	1024.0	
1753286938.725218	ea-a8-95-89-ce-e8	33-33-00-00-00-0b	udp	5353	1	8.21	6.89	768.0	
1753286938.7270744	d6-56-07-be-e5-30	33-33-00-00-00-0b	udp	5353	1	8.58	1.64	768.0	
1753286940.4481938	3e-f3-99-7f-4a-63	33-33-00-00-00-0b	udp	5353	4	3.48	2.16	1024.0	

Figura 22: flussi

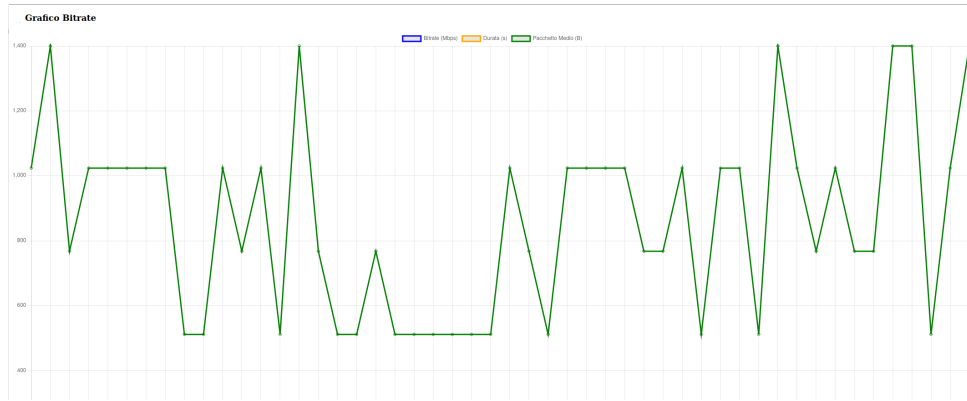


Figura 23: Grafico

Questa interfaccia nel complesso consente il monitoraggio in tempo reale dei flussi di rete SDN, favorendo l'analisi visiva e la validazione del slicing e della gestione dei servizi.

9 Conclusioni

Utilizzo di Due Controller Distinti

Per l'implementazione delle due funzionalità richieste — **Topology Slicing** e **Service Slicing** — si è scelto di adottare **due controller SDN distinti**, ciascuno con una responsabilità ben definita.

Questa decisione è stata presa per motivi di *modularità* e *chiarezza progettuale*. In particolare:

- Il **controller per il Topology Slicing** si occupa esclusivamente di instradare il traffico tra coppie di host predefinite ($H1 \leftrightarrow H3$ e $H2 \leftrightarrow H4$), imponendo vincoli di percorso attraverso regole statiche basate su indirizzi MAC. In questo modo è possibile realizzare percorsi isolati (slice) all'interno della topologia.
- Il **controller per il Service Slicing**, invece, analizza dinamicamente il traffico in base al protocollo di trasporto e alla porta di destinazione. In particolare, identifica e priorizza il traffico video (UDP su porta 9999), applicando regole che ne garantiscono un trattamento preferenziale rispetto ad altri tipi di traffico.

Inoltre, tale approccio:

- facilita il *debug* e il *monitoraggio* delle funzionalità,
- consente *test indipendenti* delle due forme di slicing,
- rende ogni controller *più semplice da mantenere o riutilizzare* in altri contesti.

In scenari SDN complessi e reali, è comune adottare architetture composte da più controller o da moduli separati all'interno di un controller centralizzato, ognuno dedicato a una funzione specifica come la gestione della topologia, la sicurezza o la qualità del servizio (QoS). L'approccio adottato in questo progetto si ispira a tale paradigma.

Nel complesso, questo progetto ci ha offerto un'opportunità concreta per mettere alla prova le potenzialità offerte dall'approccio SDN e dal Network Slicing in un contesto controllato, ma rappresentativo di scenari reali. Abbiamo visto come, grazie a Mininet e Ryu, sia possibile costruire rapidamente topologie di rete complesse e definire policy di instradamento estremamente granulari senza il peso di infrastrutture hardware fisiche. L'implementazione del Topology Slicing ci ha dimostrato che isolare segmenti di traffico non è un'operazione da grandi centri dati, ma può essere eseguita agevolmente anche su un singolo notebook, semplicemente configurando regole basate su indirizzi MAC. Allo stesso modo, il Service Slicing ha evidenziato quanto sia importante riconoscere e trattare diversamente i flussi di traffico a seconda delle applicazioni, garantendo che servizi critici come lo streaming video mantengano performance ottimali anche in condizioni di congestionamento. I test svolti — dal semplice `pingall` fino alla stesura delle flow table con `ovs-ofctl dump-flows` e all'analisi dei risultati di `iPerf` — hanno confermato che le regole OpenFlow vengono applicate correttamente e producono l'effetto desiderato: isolamento e priorità dei flussi. Questa esperienza pratica ci ha permesso di comprendere più a fondo come, dietro le quinte, il controller SDN orchestrato da Ryu reagisca agli eventi di rete in tempo reale, riscrivendo le tabelle di flusso per indirizzare il traffico in modo intelligente.